

Università degli Studi di Milano Bicocca
Scuola di Scienze
Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di Laurea in Informatica

Appunti Advanced Machine Learning

Anno Accademico 2026-2027

Contents

1 Introduction to Deep Learning **3**
- 1.1 Representation learning . . . 3
- 1.2 Deep learning . . . 4

2 Linear Models **5**
- 2.1 Score Function of Linear Models . . . 5
- 2.2 Output And Loss Function of Linear Models . . . 6
  - 2.2.1 Regression . . . 6
  - 2.2.2 Binary Classification (Hinge) . . . 7
  - 2.2.3 Binary Classification (Logistic) . . . 8
  - 2.2.4 Multi-class Classification . . . 9
- 2.3 XOR Problem in Linear Model . . . 10

3 Deep Feedforward Networks **12**
- 3.1 Anatomy of a Unit and of a Layer . . . 12
- 3.2 Multilayer Network in Matrix Form . . . 13
- 3.3 Solving XOR with NN . . . 14
- 3.4 Cost Functions . . . 15
  - 3.4.1 Gradient-Based Learning . . . 15
  - 3.4.2 Maximum Likelihood . . . 16
- 3.5 Outputs Units . . . 17
  - 3.5.1 Linear Units for Gaussian Output Distributions . . . 17
  - 3.5.2 Sigmoid Units for Bernoulli Output Distributions . . . 18
  - 3.5.3 Softmax Units for Multinoulli Output Distributions . . . 19
  - 3.5.4 Mixture Density Networks for Multimodal Targets . . . 20
- 3.6 Hidden units and Activation Functions . . . 20
  - 3.6.1 Hidden Unit as Feature Detectors . . . 20
  - 3.6.2 Choosing the hidden unit . . . 21
  - 3.6.3 Step Function . . . 21
  - 3.6.4 Sigmoid and Hyperbolic Tangent Units . . . 22
  - 3.6.5 Rectified Linear Units And Their Generalizations . . . 22
- 3.7 Architecture Design . . . 24
  - 3.7.1 Universal Approximation Properties and Depth . . . 24
  - 3.7.2 Two sigmoid units in 2D . . . 28
  - 3.7.3 Deep vs Shallow . . . 32

4 Optimization Algorithms **34**

5 Notation **35**

Chapter 1

Introduction to Deep Learning

Deep learning is a specialized subset of machine learning centered on multilayer neural networks, whose architectures are conceptually inspired by the structure of the human brain. Rather than relying on hand-crafted features, deep learning operates within the paradigm of *representation learning*, enabling models to automatically discover hierarchies of abstract features built from simpler ones. Today, deep learning underpins many of the most advanced artificial intelligence (AI) systems, driving state-of-the-art applications ranging from computer vision and generative AI to autonomous driving and robotics. As illustrated in Figure 1.1, deep learning represents a specific, highly expressive branch within the broader scope of artificial intelligence.

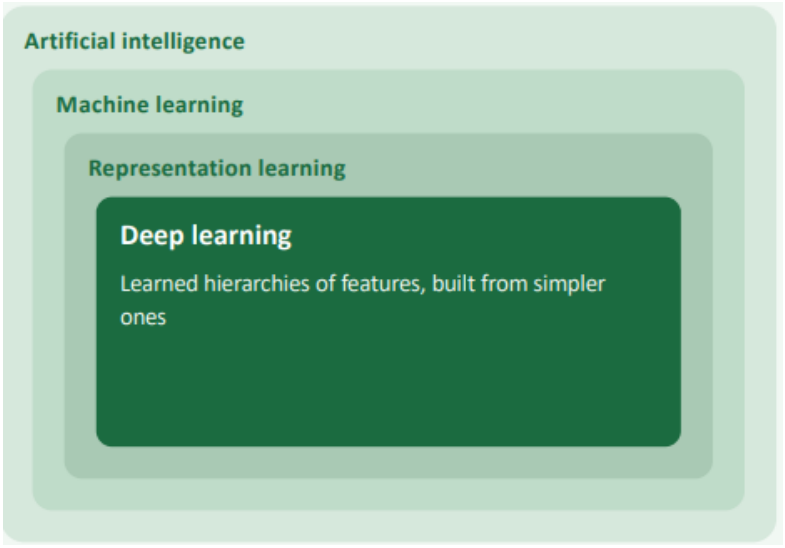


Figure 1.1: Hierarchical positioning of Deep Learning

1.1 Representation learning

What really distinguishes deep learning from traditional machine learning is its ability to perform *representation learning*. In traditional machine learning, the model is provided with a set of

features that are manually engineered by domain experts. In contrast, deep learning models automatically learn to extract relevant features from raw data with representation learning. In figure 1.2 we can see four different approaches to the task of representation. The main goal of

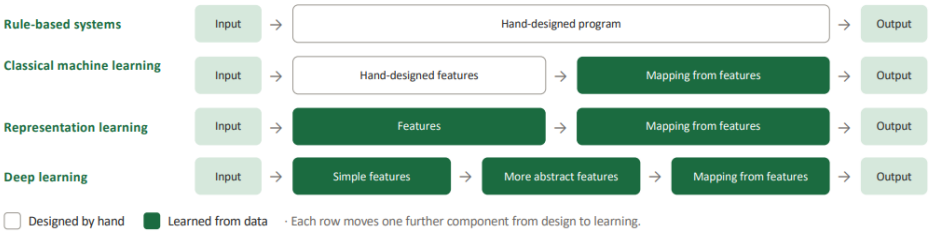


Figure 1.2: Four approaches to representation learning

representation learning is to identify the **factor of variations** that explain the observed data (that sometimes are not directly observable). They might be seen as concepts or abstraction that help us make sense of the rich variability in the data.

1.2 Deep learning

Deep learning uses a hierarchical approach to representation learning, allowing models to obtain features by introducing representations that are expressed in terms of other simpler representations. These architectures frequently leverage *distributed representations*, where concepts are encoded through the joint, many-to-many activity of multiple processing units rather than isolated nodes, significantly improving efficiency and generalization.

Chapter 2

Linear Models

In order to understand the power and the importance of Feedforward Networks, we first need to understand the limitations of linear models. Linear models are a class of models that make predictions based on a linear combination of the input features. They are simple and efficient, but they have limitations in capturing complex relationships in data. In Table 2.1, we summarize the main types of linear models, their output functions, output ranges, and loss functions.

Task	**Score**	**Output function**	**Output range**	**Loss**
Regression	$z = \mathbf{w}^\top \mathbf{x} + b$	identity: $\hat{y} = z$	$\mathbb{R}$	$(\hat{y} - y)^2$
Binary (hinge)	$z = \mathbf{w}^\top \mathbf{x} + b$	$\hat{y} = \text{sign}(z)$	$\{-1, +1\}$	$\max(0, 1 - yz)$
Binary (logistic)	$z = \mathbf{w}^\top \mathbf{x} + b$	$\hat{y} = \sigma(z)$	$(0, 1)$	$-y \log \hat{y} - (1 - y) \log(1 - \hat{y})$
Multi-class	$z = \mathbf{W}^\top \mathbf{x} + \mathbf{b}$	$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$	probability vectors	$-\log \hat{y}_y$

Table 2.1: Models with a Linear Score

2.1 Score Function of Linear Models

Every task solved with a linear model relies on a score function that computes a linear combination of the input features. The score (often denoted as z or referred to as *logits* in classification) represents the initial, raw evidence or confidence calculated by the model before any final mapping. For regression and binary classification tasks, given a single input vector $\mathbf{x} \in \mathbb{R}^d$, the score is a scalar computed as a weighted sum plus a bias:

$$z = \mathbf{w}^\top \mathbf{x} + b \tag{2.1}$$

Geometrically, this score defines a single hyperplane in the input space that serves as the decision boundary ($z = 0$) or predicts a continuous value (in the case of regression).
For multi-class tasks with k classes, the model computes a vector of scores rather than a single scalar. Using matrix notation for a weight matrix $\mathbf{W}$ and a bias vector $\mathbf{b}$:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} \tag{2.2}$$

In this multi-class setting, the model evaluates multiple scores simultaneously, where the boundaries between classes are formed where individual score components equate ($z_i = z_j$).

2.2 Output And Loss Function of Linear Models

The output and loss functions are different for each task, the first maps the raw score to the final prediction and the second measures the error between the prediction $\hat{y}$ and the true target y in order to be able to update the model parameters.

2.2.1 Regression

In regression, the output function is the identity function, meaning the raw score is used directly as the prediction:

$$\hat{y} = z \tag{2.3}$$

So the output range is the entire set of real numbers ($\mathbb{R}$). In Fig. 2.1 there is an example of a linear regression in 2d (one feature).

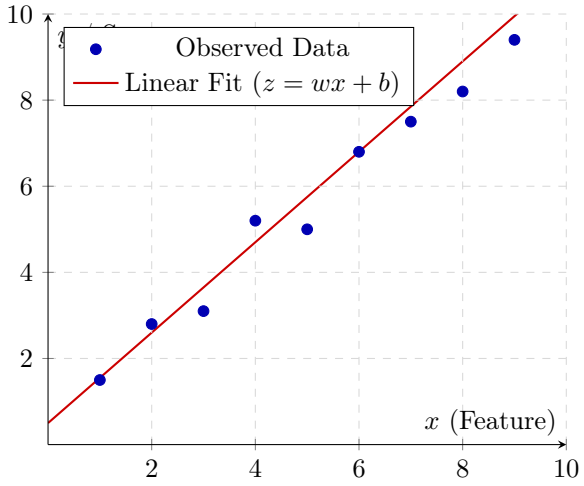


Figure 2.1: Geometric representation of 2D linear regression.

The loss function for regression is the **Squared Error**:

$$l(\hat{y}, y) = (\hat{y} - y)^2 \tag{2.4}$$

that means that the loss grows quadratically with the error, which can be sensitive to outliers. so it's often used the **Absolute Error**:

$$l(\hat{y}, y) = |\hat{y} - y| \tag{2.5}$$

that grows only linearly.
In figure 2.2 we can see a comparison between the two loss functions if we take tre example ($\hat{y}$,y): (3,5;3)(2;3)(5;3). As we can se the squared error is geometrically a parabola and the absolute error is a V shape.

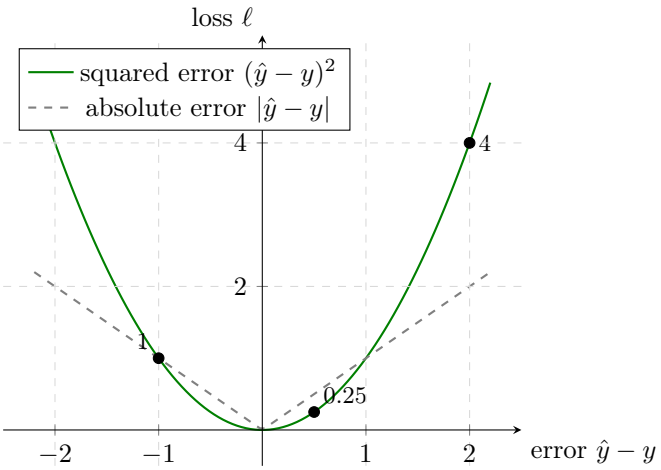


Figure 2.2: Comparison between squared error loss and absolute error loss for linear regression.

2.2.2 Binary Classification (Hinge)

In this type of classification task (e.g Support Vector Machines), the output function is the sign function, which maps the raw score to either -1 or +1:

$$\hat{y} = \text{sign}(z) \tag{2.6}$$

where

$$\text{sign}(z) = \begin{cases} +1 & \text{if } z > 0 \\ 0 & \text{if } z = 0 \\ -1 & \text{if } z < 0 \end{cases}$$

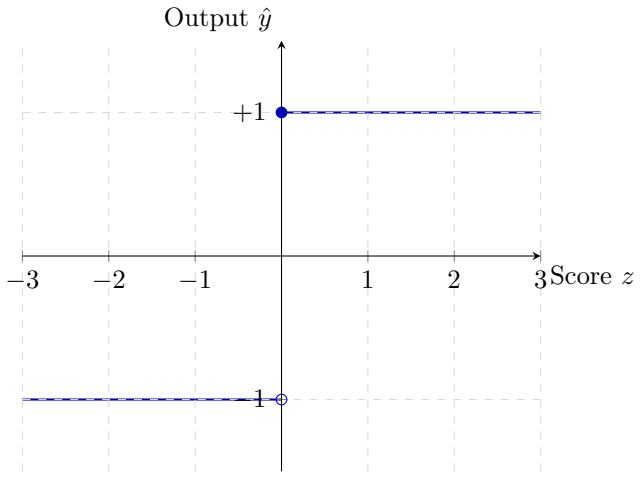


Figure 2.3: Output function for SVM / Hinge loss classification: $\hat{y} = \text{sign}(z)$.

The loss function for binary classification with hinge loss is defined as:

$$l(\hat{y}, y) = \max(0, 1 - yz) \tag{2.7}$$

This function identifies three cases:

1. $yz < 0$: the prediction is wrong and the loss grows linearly.
2. $0 \leq yz < 1$: the prediction is correct, but it is too close to the decision boundary.
3. $yz \geq 1$: the prediction is correct and sufficiently far from the boundary, so the loss is zero.

In the $2^n d$ case we have a value of z equal to 0 or close to it, so a very small variation (i.e. a noise) could lead to the opposite classification, so the loss isn't 0 even if the prediction is correct. This is the geometric intuition behind margin maximization: the model is penalized only when the prediction is either wrong or not confident enough. In Fig 2.4 there is a raffiguration of what written above if we consider three cases of (y;z): (+1;3)(+1;0,4)(-1;2).

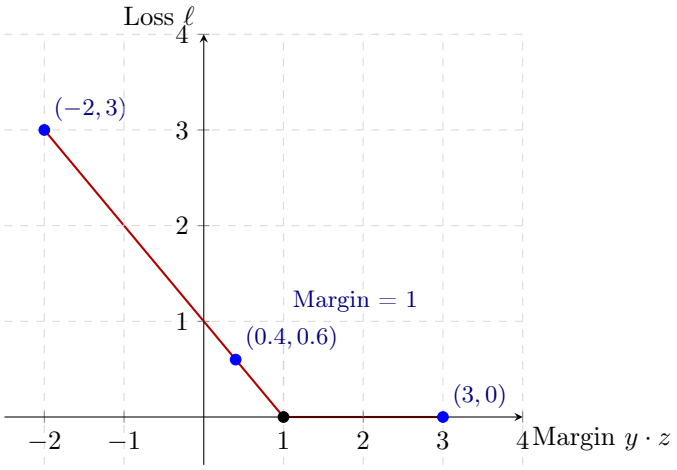


Figure 2.4: Geometric representation of the Hinge Loss function with specific examples.

2.2.3 Binary Classification (Logistic)

In this task (e.g. Logistic Regression) the goal is to predict a discrete or categorical outcome. The output function is the **sigmoid**:

$$\hat{y} = \sigma(z) \tag{2.8}$$

where σ is:

$$\sigma = \frac{1}{1 + e^{-z}}$$

which maps the raw score in the range of [0,1], as shown in Fig 2.5

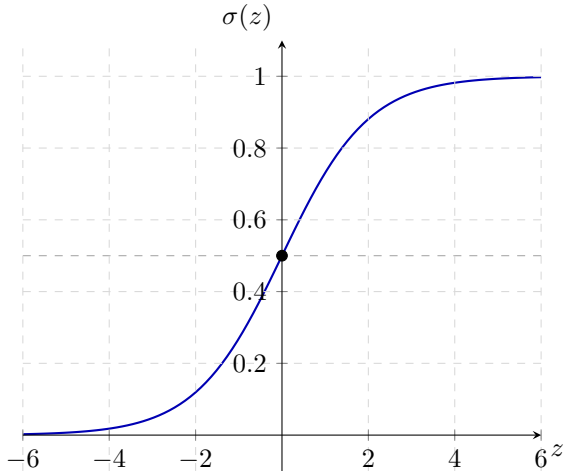


Figure 2.5: Geometric representation of the sigmoid function.

The loss fuction is the **Binary Cross-Entropy Loss**:

$$\mathcal{L}(y, \hat{y}) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y}) \tag{2.9}$$

The model outputs a probability that the class is 1, so when z is large the loss is small; The smaller zeta becomes, the greater the loss becomes. For the class $y = 0$ is mirrored. If we look at the formula 2.2.3, the loss can't be really 0.

2.2.4 Multi-class Classification

In multi-class classification tasks (e.g., Softmax Regression / Multinomial Logistic Regression), the model predicts a probability distribution across C discrete classes. The output function is the **softmax** function, which maps a vector of raw scores $\mathbf{z} = [z_1, z_2, \ldots, z_C]^T$ into a vector of probabilities $\hat{\mathbf{y}}$:

$$\hat{y}_c = \text{softmax}(\mathbf{z})_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}} \tag{2.10}$$

where each predicted probability satisfies $\hat{y}_c \in (0, 1)$ and the sum of all probabilities equals one: $\sum_{c=1}^{C} \hat{y}_c = 1$.
The loss function for multi-class classification is the **Cross-Entropy Loss**:

$$\ell = -\log \hat{y}_y \tag{2.11}$$

The softmax gives one probability per class $(\hat{y}_1, \ldots, \hat{y}_k)$ summing to 1, and only the probability of the true class y enters the loss.

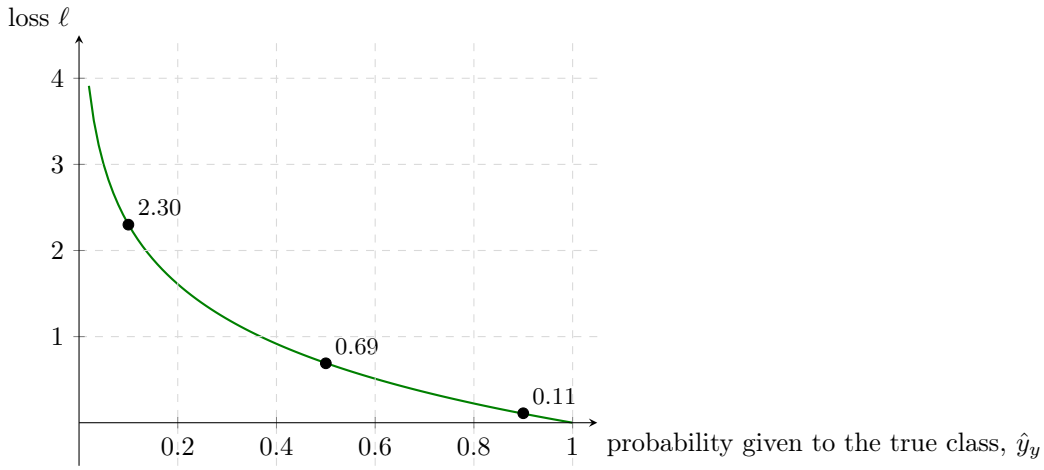


Figure 2.6: Multi-class cross-entropy loss $\ell = -\log \hat{y}_y$ with specific examples.

2.3 XOR Problem in Linear Model

Models that rely purely on a linear score followed by a fixed non-linear output function (like logistic regression) are known as *Generalized Linear Models*. Even though they apply a non-linear function (e.g., the S-shaped sigmoid curve), this function is strictly monotonic and only rescales the score. Consequently, the decision boundary,the region where the model decides to switch classes, remains a flat **hyperplane** ($z = 0$).
Consider the example in Fig 2.7, suppose a linear classifier $\hat{y} = 1 \iff \mathbf{w}^\top \mathbf{x} + b \geq 0$. Now considering the four point of the XOR:

- (0,0): $b < 0$
- (1,0): $w_1 + b \geq 0$
- (0,1): $w_2 + b \geq 0$
- (1,1): $w_1 + w_2 + b < 0$

we obtain the conditions for the parameters of the model, but if we combine the conditions for the intermediate points: $w_1 + w_2 + b + b \geq 0 \implies w_1 + w_2 + b \geq -b$ but from the first condition we know $b < 0$ so $w_1 + w_2 + b > 0$ but this is contradiction with the fourth condition. **No such w,b exists**.

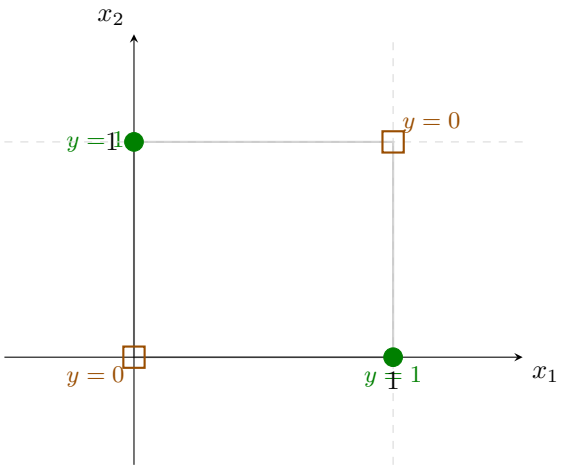


Figure 2.7: Geometric representation of the XOR problem showing linear non-separability.

The only fix to solve those type of problem it is to compute the score on new features $\phi(x)$. The **Representation** $\phi : \mathbb{R} \implies \mathbb{R}^{\mathbb{D}}$, turns the d raw inputs into D new features and then the model use those feature in the logit:

$$f(\mathbf{x}; \theta, \mathbf{w}) = \phi(\mathbf{x}; \theta)^{\top} \mathbf{w} + b$$

$\phi(x)$ can be obtained in three ways:

- **Generic** ϕ: a fixed rich map that poorly generalizes on hard problems
- **Hand designed** ϕ: written by experts, expensive and demanding
- **Learned** $\phi(\mathbf{x}; \theta)$: learned by gradient descent

Chapter 3

Deep Feedforward Networks

Also called multilayer perceptrons, a feedforward network is a composition of functions with a certain **Depth**(number of layers with parameters), **Width**(number of units dI in layer I). The goal of training is to choose θ so that

$$f(x;\theta) \approx f^*(x)$$

on new inputs, not only on training set.

3.1 Anatomy of a Unit and of a Layer

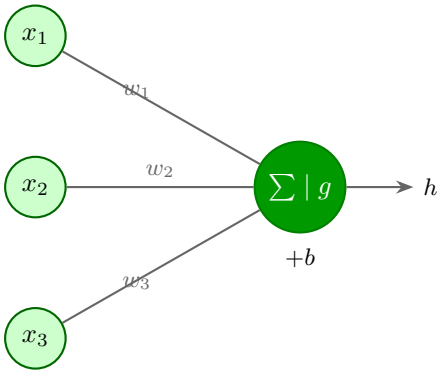
For one layer of m units (all read the same $\mathbf{x}$):

$$\mathbf{h} = g(\mathbf{W}^\top \mathbf{x} + \mathbf{b}), \quad \mathbf{x} \in \mathbb{R}^d,\ \mathbf{W} \in \mathbb{R}^{d\times m},\ \mathbf{b} \in \mathbb{R}^m \tag{3.1}$$

Column j of $\mathbf{W}$ holds the weights of unit j; g is applied to each entry separately.
$z = \mathbf{w}^\top \mathbf{x} + b$ is the **pre-activation**: a weighted sum plus bias. $h = g(z)$ is the **activation**; g is a fixed non-linear function, e.g. the ReLU $g(z) = \max\{0, z\}$.
As an example with $d = 3$ inputs and $m = 2$ ReLU units:

$$\mathbf{x} = (1, 0, 2), \quad \mathbf{W} = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b} = (0, -1)$$
$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} = (1 + 0 + 2, -1 + 0 + 2) + (0, -1) = (3, 0)$$
$$\mathbf{h} = \max\{0, \mathbf{z}\} = (3, 0)$$

For n examples at once: $\mathbf{H} = g(\mathbf{X}\mathbf{W} + \mathbf{1}\mathbf{b}^\top)$; row i of $\mathbf{H}$ is the layer output for example i.



Definition 1 (Saturation) *A unit is saturated when $|z|$ is so large that its output hardly changes: the slope $g'(z)$ is almost 0*

So when the slope is almost 0 the derivative is 0, is impossible to know from data how much the model has to change his parameters.

3.2 Multilayer Network in Matrix Form

The forward pass through an L-layer network is defined as:

$$\begin{aligned}\mathbf{h}^{(0)} &= \mathbf{x}\\ \mathbf{h}^{(l)} &= g(\mathbf{W}^{(l)\top}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}), \quad l = 1, \ldots, L-1\\ \hat{\mathbf{y}} &= o(\mathbf{W}^{(L)\top}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)})\end{aligned}$$

The last pre-activation, $\mathbf{z}^{(L)} = \mathbf{W}^{(L)\top}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$, is called the vector of **logits**; the output function o (identity, sigmoid or softmax) turns it into the prediction.

Definition 2 (Number of parameters) *Layer l has $d_{l-1} \cdot d_l$ weights and d_l biases:*

$$|\theta| = \sum_{l=1}^{L}(d_{l-1} + 1)d_l \tag{3.2}$$

Most parameters sit in the first layer. In Fig 3.1 there is an example of the calculus of parameters. The MLP is : $784 \to 256 \to 128 \to 10$. Input: $28 \times 28 = 784$ pixels. Output: 10 classes.

Layer	$\mathbf{W}^{(l)}$	biases	parameters
1	784×256	256	200 960
2	256×128	128	32 896
3 (output)	128×10	10	1 290
total			235 146

Table 3.1: Parameter count breakdown for the MNIST network architecture.

3.3 Solving XOR with NN

In the end of the last chapter we saw the limitations of linear model in the case of some problems like XOR. With NN this problem can be solved with one hidden layer as shown in Fig 3.2.

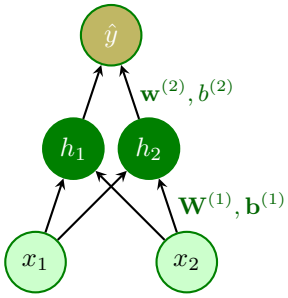


Figure 3.1: Neural network architecture for the XOR problem.

The forward pass is defined as:

$$\hat{y} = \mathbf{w}^{(2)\top} \max\left\{0, \mathbf{W}^{(1)\top}\mathbf{x} + \mathbf{b}^{(1)}\right\} + b^{(2)}$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{w}^{(2)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad b^{(2)} = 0$$

2 inputs, 2 hidden units, 1 output: $4 + 2 + 2 + 1 = 9$ parameters.

$$\underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}}_{\mathbf{X}} \longrightarrow \underbrace{\begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}}_{\mathbf{X}\mathbf{W}^{(1)}} \longrightarrow \text{add } \mathbf{b}^{(1)} \text{ to each row} \longrightarrow \underbrace{\begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}}_{} \longrightarrow \underbrace{\begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}}_{\max\{0,\cdot\}} \longrightarrow \underbrace{\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}}_{\times \mathbf{w}^{(2)} = \hat{\mathbf{y}}}$$

Figure 3.2: Step-by-step matrix representation of the XOR forward pass for an entire batch.

In $\mathbf{x}$-space no line separates the classes. In $\mathbf{h}$-space the two positive points coincide and a line does:

$$h_1 - 2h_2 = \frac{1}{2} \tag{3.3}$$

The output layer is still a **linear score**, computed on $\mathbf{h} = \phi(\mathbf{x})$ instead of $\mathbf{x}$.

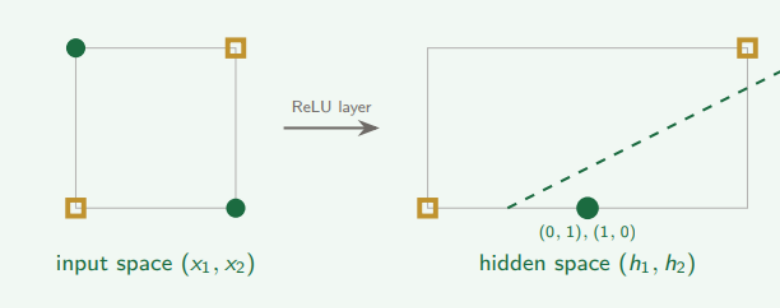


Figure 3.3: Difference between x-space and h-space

3.4 Cost Functions

The cost function is a mathematical formula that measures the error between the network's predicted output and the actual target values across a dataset.

Remark 1 *Some sources consider loss and cost to be synonyms, while others consider cost to refer to the entire dataset or a batch of data.*

3.4.1 Gradient-Based Learning

As with linear models, the gradient-based learning is a foundamental training paradigm where model parameters are iteratively adjusted to minimize a cost function $J(\theta)$ using the **gradient descent algorithm**:

$$\theta \leftarrow \theta - \eta \nabla_\theta J \tag{3.4}$$

where η is the learning rate.
What changes in NN is the fact that the non-linearities make **J non-convex**, as shown in Fig 3.4, so gradient descent may stop in a minimum that is not the best one, and where it stops depends on the starting θ. In conclusion, the initial parameters θ must be *small* in order to start in a non-saturated range, and **random** because otherwise every hidden unit would compute the same thing and receive the same update (random values break this symmetry).

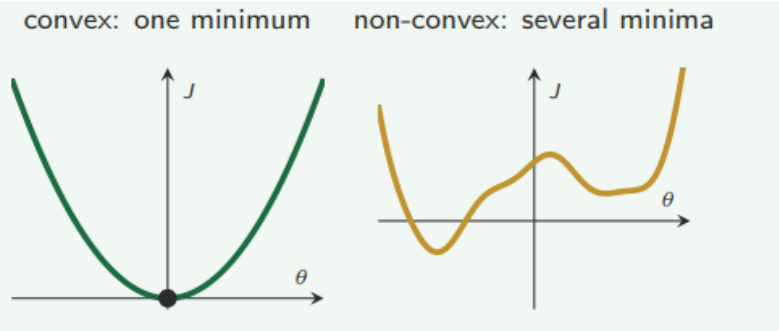


Figure 3.4: Difference between convex and non-convex

One optimization of the gradient-descent algorithm is the

Definition 3 (Minibatch Stochastic Gradient Descent) *The algorithm is repeated until the validation error stops improving. At each step, the algorithm:*

1. *Picks m training examples at random to form a minibatch $\mathcal{B}$.*
2. *Averages their gradients to compute $\hat{g} = \frac{1}{m}\sum_{i\in\mathcal{B}} \nabla_\theta \ell(f(\mathbf{x}^{(i)};\theta), y^{(i)})$, where individual gradients are computed via back-propagation.*
3. *Takes an optimization step: $\theta \leftarrow \theta - \eta\hat{g}$ using learning rate η.*

Here, $\hat{g}$ is a noisy but unbiased estimate of the true gradient $\nabla_\theta J$. A complete pass over all n training examples is called an epoch. The noise of the estimate decreases as $1/\sqrt{m}$, with typical batch sizes ranging between 32 and 512 ($32 \leq m \leq 512$).

3.4.2 Maximum Likelihood

The cost function comes from maximum likelihood, the network defines a **probability distribution** $p_{model}(y|\mathbf{x};\theta)$ over the possible targets. An advantage of this approach is that is not necessary to design cost functions for each model.

Definition 4 (Maximum Likelihood) *choose θ so that the correct answers get high probability. In other words, minimize the average negative log-probability:*

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \log p_{model}(y^{(i)} \mid \mathbf{x}^{(i)};\theta) \tag{3.5}$$

An other form of this function is

$$J(\boldsymbol{\theta}) = -\mathbb{E}_{\mathbf{x},\mathbf{y}\sim\hat{p}_{\text{data}}} \log p_{\text{model}}(\boldsymbol{y} \mid \boldsymbol{x}). \tag{3.7}$$

where $\hat{p}_{\text{data}}$ gives probability 1/n to each training pair. This is also called **Cross-Entropy**. The negativity helps to avoid the problem of the saturation of the units of the NN.

Derivation: A Gaussian Output Gives the Squared Error

A practical application of this principle is choosing a Gaussian output distribution for the network, assuming the target is the network output plus Gaussian noise with variance s^2:

$$p_{\text{model}}(y \mid \mathbf{x}) = \mathcal{N}(y; f(\mathbf{x};\boldsymbol{\theta}), s^2) = \frac{1}{\sqrt{2\pi s^2}} \exp\left(-\frac{(y - f(\mathbf{x};\boldsymbol{\theta}))^2}{2s^2}\right) \tag{3.8}$$

Taking the negative log-probability yields:

$$-\log p_{\text{model}}(y \mid \mathbf{x}) = \frac{1}{2s^2}(y - f(\mathbf{x};\boldsymbol{\theta}))^2 + \frac{1}{2}\log(2\pi s^2) \tag{3.9}$$

Since the last term does not depend on $\boldsymbol{\theta}$, minimizing the average negative log-likelihood over the dataset leads directly to the Mean Squared Error cost function:

$$J(\boldsymbol{\theta}) = \frac{1}{2s^2} \cdot \frac{1}{n}\sum_i \left(y^{(i)} - f(\mathbf{x}^{(i)};\boldsymbol{\theta})\right)^2 + \text{const} \tag{3.10}$$

Thus, under a Gaussian assumption, maximum likelihood is equivalent to minimizing the squared error.

Mean absolute or squared error often lead to poor results when used with gradient-based optimization due to saturation.

3.5 Outputs Units

The choice of cost function is tightly coupled with the choice of output unit. The feedforward network provides a set of hidden featured defined by $\mathbf{h} = f(\mathbf{x}; \theta)$, and the role of output layer is then to provide some additional transformation from the features to complete the task.

3.5.1 Linear Units for Gaussian Output Distributions

Given features $\mathbf{h}$, a layer of linear output units produces a vector $\hat{\mathbf{y}} = \mathbf{W}^T\mathbf{h} + \mathbf{b}$. The advantage of a linear unit is that it never saturates because the slope is always 1. They are used to produce the mean of a conditional Gaussian distribution:

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{y}; \hat{\boldsymbol{y}}, I). \tag{3.11}$$

As we saw before, this leads to a **squared error** for the loss.
Furthermore, the network can output two numbers: the mean $\hat{y}$ and a value v for the spread. Since the variance must be strictly positive, we set:

$$s^2 = \zeta(v) = \log(1 + e^v) \quad (\text{always} > 0) \tag{3.12}$$

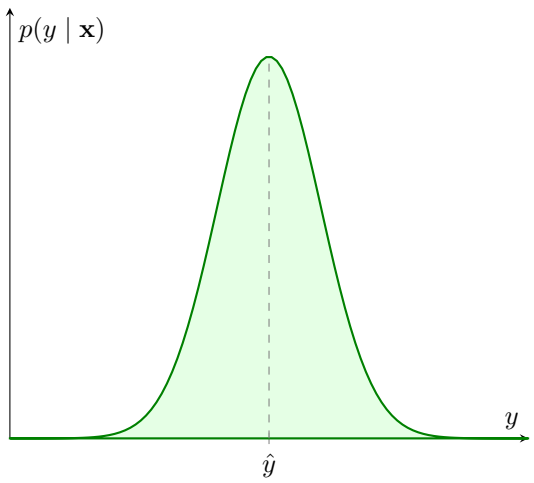


Figure 3.5: Gaussian Distribution $p(y \mid \mathbf{x})$ centred on $\hat{y}$.

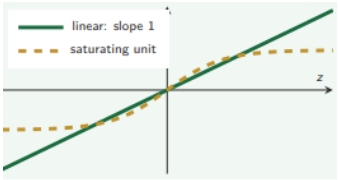


Figure 3.6: Graphical rapresentation of output unit

3.5.2 Sigmoid Units for Bernoulli Output Distributions

Usefull in classification problems with two classes. A Bernoulli distribution is defined by just a single number in the interval [0,1]. The NN needs to predict only $P(y = 1|\mathbf{x})$. The sigmoid is motivated by constructing an unnormalized probability distribution $\tilde{P}(y)$ which does not sums to 1.

$$\begin{aligned}
\log \tilde{P}(y) &= yz, \\
\tilde{P}(y) &= \exp(yz), \\
P(y) &= \frac{\exp(yz)}{\sum_{y'=0}^{1} \exp(y'z)}, \\
P(y) &= \sigma((2y-1)z).
\end{aligned}$$

The variabile z that defines a distribution over binary variables is called **logit**. The loss function for maximum likelihood learning of a Bernoulli parametrized by sigmoid is the **Binary Cross-Entropy**:

$$\begin{aligned}
J(\boldsymbol{\theta}) &= -\log P(y \mid \boldsymbol{x}) \\
&= -\log \sigma((2y-1)z) \\
&= \zeta((1-2y)z). \quad \text{(softplus)}
\end{aligned}$$

Using the softplus version, in Fig 3.8 we can see that *saturation* occurs only when the model already has the right answer. When z has the wrong sign, the argument of the softplus may be simplified to $|z|$ and so the derivative with respect to z asymptotes to $sign(z)$, meaning that the gradient won't vanish at all. If we look at the other loss like the **Squared Error**, the loss can saturate anytime $\sigma(z)$ saturates (The sigmoid activation function saturates to 0 when z becomes very negative and saturates to 1 when z becomes very positive)

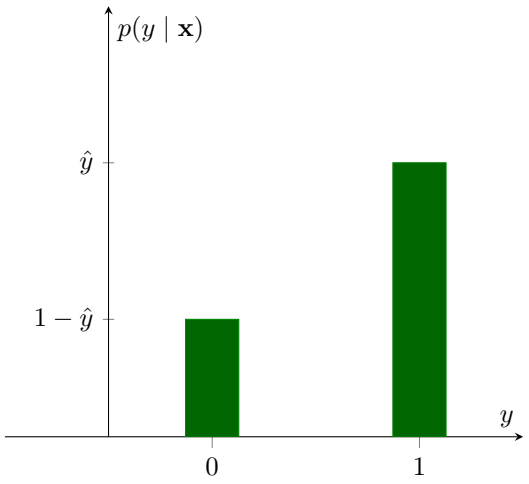


Figure 3.7: Bernoulli distribution $p(y \mid \mathbf{x})$ for binary classification.

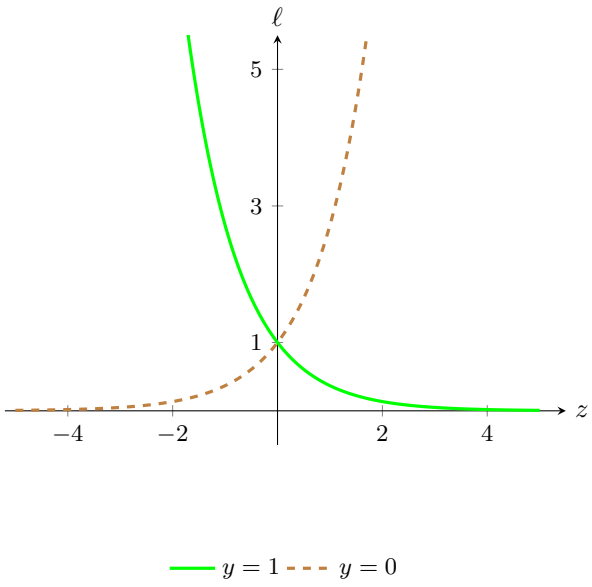


Figure 3.8: Loss functions for $y = 1$ and $y = 0$.

3.5.3 Softmax Units for Multinoulli Output Distributions

Useful in classification problems with k classes (categorical outputs). The network needs to produce a vector $\hat{\mathbf{y}}$ representing the probability distribution over k different classes, where each element is in $[0, 1]$ and the entire vector sums to 1. The softmax function turns logits $\mathbf{z} \in \mathbb{R}^k$ into probabilities:

$$\text{softmax}(\mathbf{z})i = \frac{e^{z_i}}{\sum j = 1^k e^{z_j}}, \quad i = 1, \ldots, k$$

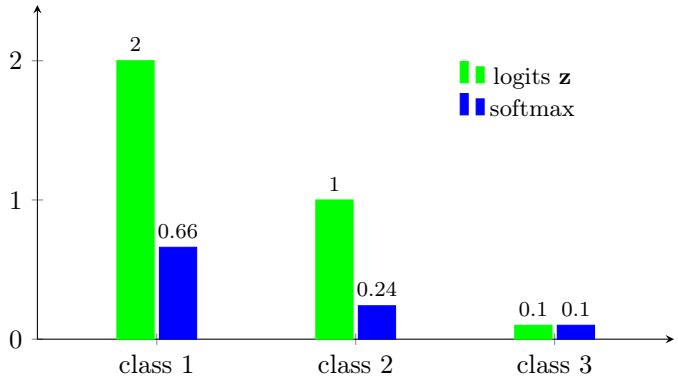
The loss function for the true class y is the negative log-likelihood:

$$\ell = -\log \text{softmax}(\mathbf{z})y \ = -z_y + \log \sum j e^{z_j}.$$

The first term pushes the correct logit up, while the second term pushes all logits down (the largest one the most). The gradient of the loss with respect to the logits is:

$$\nabla_{\mathbf{z}} \ell = \text{softmax}(\mathbf{z}) - \mathbf{e}_y$$

which represents the prediction minus the one-hot target vector.



3.5.4 Mixture Density Networks for Multimodal Targets

Useful for modeling multimodal targets by representing the conditional probability distribution as a weighted sum of K Gaussian bell curves:

$$p(y \mid \mathbf{x}) = \sum_{c=1}^{K} \pi_c(\mathbf{x}) \mathcal{N}(y; \mu_c(\mathbf{x}), s_c^2(\mathbf{x}))$$

The network outputs three groups of parameters for each component c:

- **Weights** $\pi_c(\mathbf{x})$: constrained via a softmax over K (positive and summing to 1).
- **Centres** $\mu_c(\mathbf{x})$: produced by standard linear units.
- **Spreads** $s_c^2(\mathbf{x})$: constrained to be positive using softplus (ζ) or exponential functions.

3.6 Hidden units and Activation Functions

3.6.1 Hidden Unit as Feature Detectors

If we consider the *trigonometric* form of the pre-activation function $z = \mathbf{w^T x} + b = ||w||\,||x|| cos\theta + b$ where θ is the angle between $\mathbf{w}$ and $\mathbf{x}$. For inputs of similar length, z is largest when $\mathbf{x}$ has the same direction of $\mathbf{w}$, so each hidden unit is a learned **template**. The bias estabilish how much strict must be the correspondence for the unit to be activated. In the example showed in Fig 3.9, the first layer units' become detectors of strokes (es. vertical lines)

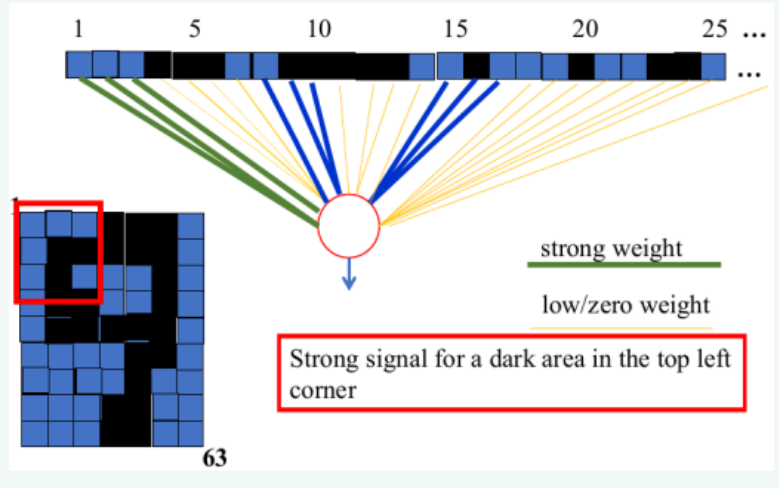


Figure 3.9: An 8x8 digit image flattened into 64 inputs

The limitation is that a unit is tied to fixed pixel position, so the same stroke elsewhere needs another unit.

3.6.2 Choosing the hidden unit

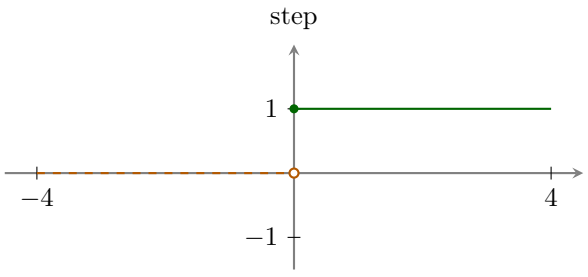
Every hidden unit can be described as accepting a vector of inputs $\mathbf{x}$, computing an affine transformation $\mathbf{z} = \mathbf{W}^{\mathbf{T}}\mathbf{x} + \mathbf{b}$, and then applying an element-wise non linear function $g(\mathbf{z})$ (*activation function*). So units differ only in the activation function, g needs three rules:

- non linearity
- non zero slope $g^i(\mathbf{z})$
- cheap computation

Which g works best is hard to predict, so it is obtained by trial and error.

3.6.3 Step Function

Easiest example of activation function, the slope is 0 everywhere, so the gradient descent gets no signal.



3.6.4 Sigmoid and Hyperbolic Tangent Units

This two type of units are grouped together because they're functions are closely related: $tanh(z) = 2\sigma(2z) - 1$. The use of the sigmoid as activation function is discouraged due to the fact that it saturates when z is very positive or very negative (Fig 3.10). When the sigmoid must be used, the hyperbolic tangent activation typically performs bettern than the logistic sigmoid.

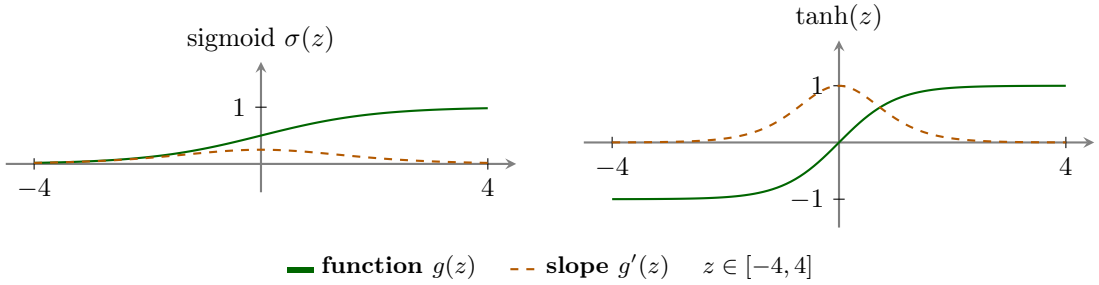


Figure 3.10: Sigmoid and hyperbolic tangent activation functions and their derivatives.

3.6.5 Rectified Linear Units And Their Generalizations

Those units use the activation function $g(z) = max(0, z)$. Easy to optimize because they are so similar to linear units with the only difference that ReLU outputs zero across half its domain. When active, the unit passes the error signal back **unchanged** (slope 1), no saturation on the positive side.
ReLU isn't differentiable[1] at 0, so the software return 1 or 0, in any case most of the time when we work with float types it's rare to have a perfect 0 value.
When initializing the parameters, it can be a good practice to set all elements of **b** to a small positive value (i.e. 0.1) in order to avoid the dead units. Infact if z $<$ 0 for every training example, the unit always outputs 0, its slope is always 0, and its weights never change again. There are some **generalization** that are based on using a nonzero slope α_i when $z_i < 0$:

$$h_i = g(z, \alpha)_i \ = max(0, z_i) + \alpha_i min(0, z_i) \tag{3.13}$$

and they are:

- Absolute value: $\alpha = -1$ and so $g(z) = |z|$, used for object recognition from images
- Leaky ReLU: $\alpha = 0,01$
- Parametric ReLU α is learned, the data choose the leak.

One further generalization is the **Maxout units**, instead of applying an element-wise function $g(z)$, maxout units divide **z** into groups of k values. A maxout unit can learn a piecewise linear, convex function with up to k pieces. It of course has a k times more weights per unit.

$$h = \max_{j=1,\dots,k} \left(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j \right) \tag{3.14}$$

[1] a function is differentiable if can be locally approximated by a linear function

Activations in Modern Networks

Modern networks use a variation of the ReLU that try to minimize the number of dead units, in Fig 3.11 there is a summarize of the 3 variations.
The difference between ReLU and GELU is the fact that GELU multiplies z by a soft gate while ReLU uses a hard gate. Thanks to this GELU gains fewer dead units.
Infact **ReLU** is defined as:

$$\text{ReLU}(z) = z \cdot \mathbb{1}[z > 0] \tag{3.15}$$

and **GELU** is defined as:

$$\text{GELU}(z) = z\Phi(z), \quad \Phi(z) = P(Z \leq z), \quad Z \sim \mathcal{N}(0,1) \tag{3.16}$$

SwiGLU (Swish-Gated LU) the idea is similar as GELU, with σ in place of Φ: $\text{SiLU}(z) = z\sigma(z)$. In GELU and SiLU the gate depends on z **itself**. In SwiGLU a **second linear map** of the input computes it:

$$\boldsymbol{h} = \underbrace{\left(\boldsymbol{W}_a^\top \boldsymbol{x}\right)}_{\text{content } \boldsymbol{a}} \odot \underbrace{\text{SiLU}\left(\boldsymbol{W}_b^\top \boldsymbol{x}\right)}_{\text{gate } \boldsymbol{s}}, \qquad \hat{\boldsymbol{y}} = \boldsymbol{W}_2^\top \boldsymbol{h} \tag{3.17}$$

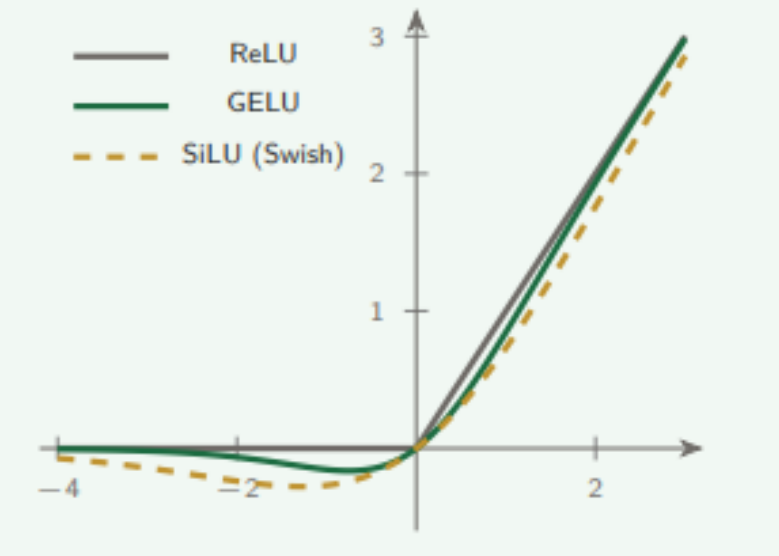


Figure 3.11: Comparison between activation functions

In conclusion, when using ReLU or a variant, the rule is to pick random and small weights and small biases (i. e. 0,01). It is important not to start with weights equal to 0 because other wise the gradient would **vanish**. It is important not to pick a too large weights neither because otherwise the gradient would **explodes**. THe idea is to choose them from a set $Var(w)$ ruled by a variance value, there are two main formulations:

- Glorot: $\frac{2}{n_{in}+n_{out}}$
- He: $\frac{2}{n_{in}}$

where n is the number of inputs per unit.

3.7 Architecture Design

The architecture of a network is defined by the number of the units the it has and how they are connected to each other. A group of units that works in linear combinations is called **layer**, most of architecture arrange these layers in a chain structure in which each layer is a function of the layer that preceded it.
In this structure, the first layer is given by $\mathbf{h}^{(1)} = g^{(1)}\left(\mathbf{W}^{(1)\top}\mathbf{x} + \mathbf{b}^{(1)}\right)$ and the second layer is given by $\mathbf{h}^{(2)} = g^{(2)}\left(\mathbf{W}^{(2)\top}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right)$ and so on. This mean that in this chain-based architectures, the main architectural considerations are choosing the depth of the network and the width of each layer. Generally speaking, one hidden layer is sufficient to fit the training set, but it isn't as optimized as a deeper network. The ideal network architecture for a task must be found via experimentation with the validation set error.

3.7.1 Universal Approximation Properties and Depth

Feedforward networks with hidden layers provide a universal approximation framework:

Theorem 1 (Universal Approximation Theorem) *For any continuous f on a closed and bounded region of $\mathbb{R}^d$ and any tolerance $\varepsilon > 0$, there is a network with **one hidden layer** of N units and a linear output,*

$$\hat{f}(\boldsymbol{x}) = \sum_{i=1}^{N} v_i \, g\left(\boldsymbol{w}_i^{\top}\boldsymbol{x} + b_i\right),$$

with $|f(\boldsymbol{x}) - \hat{f}(\boldsymbol{x})| < \varepsilon$ for every $\boldsymbol{x}$ in the region. Valid for sigmoid, tanh, ReLU: any continuous g that is not a polynomial.

In other words, one hidden layer can represent any continuous function as accurately as we want. This theorem means that regardless of what function we are trying to learn, we know that a large MLP will be able to represent it. We are not guaranteed that the training algorithm will be able to learn that function.

Sigmoid Application of Universal Approximation Theorem

A hidden unit with one input computes $\sigma(wx + b) = \sigma(k(x - a))$ where $k = w$ is the **weight** that sets the steepness and $a = -\frac{b}{w}$ is the bias that describe where the step happens.
In Fig 3.12 it is shown that a large weight turns the sigmoid into a **step**.

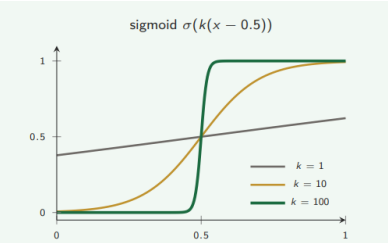


Figure 3.12: Steep sigmoid

If we take a network with one input and two hidden sigmoid, each unit is a **soft step** and their difference is a **bump** (Fig 3.13).

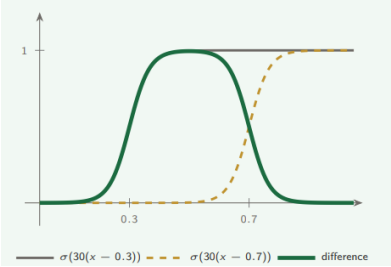


Figure 3.13: Two sigmoid units build a bump

So in conclusion the approximation can be done with the sums of bumps, the recipe is:

1. Split [0,1] into N intervals of width 1/N
2. Put a bump of height $h_i = f(interval\ centre)\ \forall\ interval\ i$
3. Add the bumps

So formally in the network:

$$\hat{f}(x) = h_1 + \sum_{i=2}^{N} (h_i - h_{i-1})\,\sigma(k(x - a_i)) \tag{3.18}$$

As we can see steps are shared within the bumps(Fig 3.14), so the sum needs only $N-1$ sigmoid unit plus a bias.

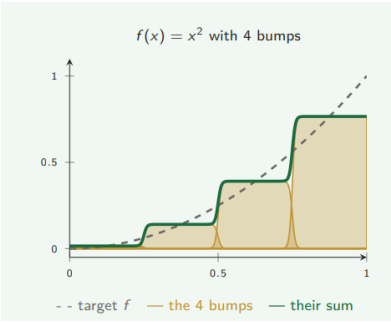


Figure 3.14: Two sigmoid units build a bump

The approximation is corrected then by modifying the number of units and the parameters, in Fig 3.15 we can see that with a moderate k the steps overlap and the sum becomes a smooth curve.

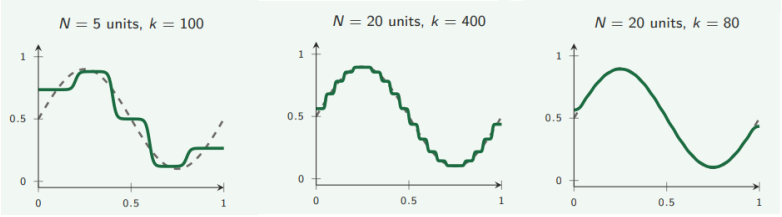


Figure 3.15: Smooth curve evolution

By the same logic we can also represent **Peak** and **Kink** (Fig 3.16)

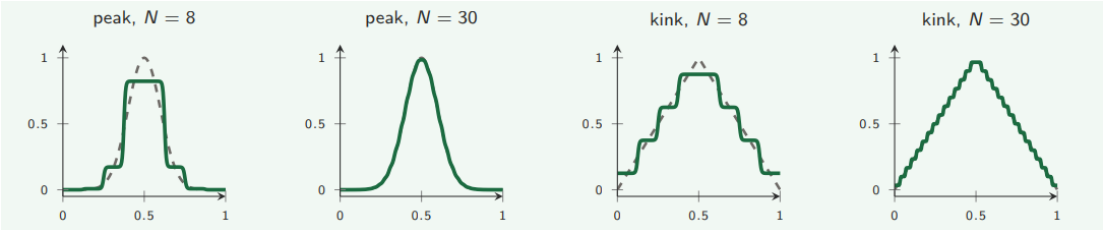


Figure 3.16: Peak and Kink evolution

For the case of **Jumps** when the function is discontinuos, the bigger is k and the narrower the zone of the error is, but it is never zero for definition, because the sum of sigmoid is continuous as shown in Fig 3.17

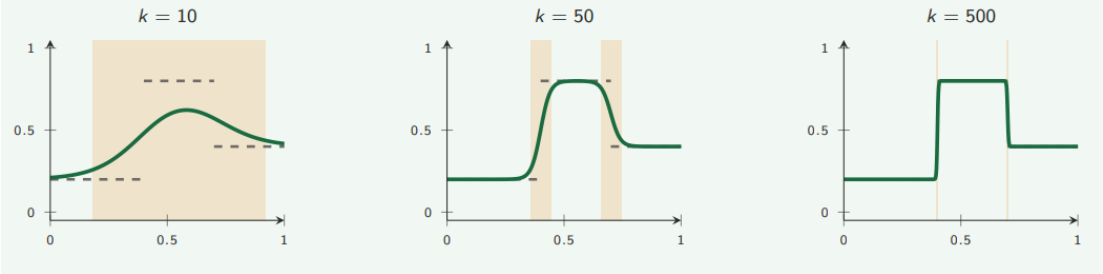


Figure 3.17: Jump approximation

ReLU Application of Universal Approximation Theorem

The foundational principle for function approximation with ReLU remains equivalent to the sigmoid case: decomposing the input space and summing elementary local building blocks to construct the target function. However, the geometric properties of these building blocks differ due to the piecewise linear nature of this function. Unlike the sigmoid function, which generates smooth steps and bell-shaped curves, a single hidden unit with ReLU computes a linear **ramp**:

$$h(x) = \max(0, wx + b) \tag{3.19}$$

where the weight w controls the slope and the bias b determines the activation threshold (the elbow point).

- Subtracting **three** shifted ReLU units with opposite slopes yields a **triangular bump** (tent function).
- Summing these triangular bumps creates a piecewise linear spline that fits any continuous 1D target function.

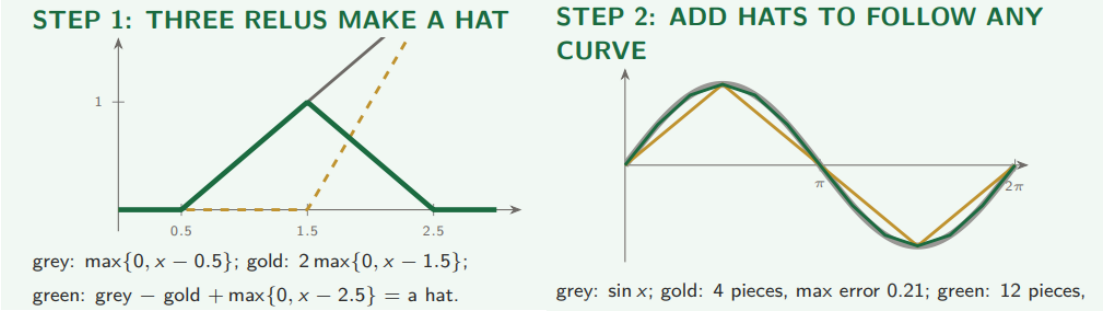


Figure 3.18: Approximation with ReLU

Two-dimensional approximation

In one dimension, a sigmoid unit produces a smooth step. In two dimensions, the same unit

$$h(\mathbf{x}) = \sigma(w_1 x_1 + w_2 x_2 + b) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}},$$

with input $\mathbf{x} = (x_1, x_2)$, can be seen as that step stretched along the new dimension. Plotted over the (x_1, x_2) plane, it gives a surface that is low on one side of a line and high on the other. We call this surface a **wall**. The unit depends on $\mathbf{x}$ only through the scalar $\mathbf{w}^\top \mathbf{x} + b$, so h is constant on every line

$$w_1 x_1 + w_2 x_2 + b = c.$$

Indeed, if we move by a small displacement $(\Delta x_1, \Delta x_2)$ along such a line, the quantity $\mathbf{w}^\top \mathbf{x} + b$ does not change, hence $\mathbf{w}^\top \Delta \mathbf{x} = 0$. This means that *the level lines are perpendicular to* $\mathbf{w}$, and $\mathbf{w}$ points in the direction in which h grows fastest, i.e. the direction in which the wall rises. The parameters play the following roles:

- The **direction** of $\mathbf{w}$ sets the orientation of the wall. For example, $\mathbf{w} = (1, 0)$ gives vertical level lines, $\mathbf{w} = (0, 1)$ gives horizontal ones, and $\mathbf{w} = (1, 1)$ gives diagonal lines $x_1 + x_2 =$ const.
- The **norm** $\|\mathbf{w}\|$ sets the steepness: large weights give a nearly vertical transition, small weights a gentle one.
- The **bias** b translates the wall without changing its orientation. The transition, where $h = 1/2$, lies on the line $\mathbf{w}^\top \mathbf{x} + b = 0$.

The problem with a wall is that it is infinitely long along the direction in which h does not change, so it cannot be a localized bump. To enclose a bounded region we need to combine several walls with *different orientations*: the result is a **tower**. A concrete construction for the square $[-1, 1]^2$ uses four hidden units with steepness k:

$$s(\mathbf{x}) = \sigma\big(k(x_1 + 1)\big) + \sigma\big(-k(x_1 - 1)\big) + \sigma\big(k(x_2 + 1)\big) + \sigma\big(-k(x_2 - 1)\big).$$

For large k, $s \approx 4$ inside the square, $s \approx 3$ outside it but beyond a single side, and $s \approx 2$ in the outer corner regions. Summing the walls alone therefore does not give a uniform plateau. Adding an output unit that thresholds the sum,

$$T(\mathbf{x}) = \sigma\big(k\,(s(\mathbf{x}) - 3.5)\big),$$

yields $T \approx 1$ inside the square and $T \approx 0$ elsewhere, which is a tower. This construction uses five units arranged in two layers, since the second nonlinearity is what produces the flat top. With a single hidden layer and a linear readout, a tower can still be approximated, but with more units, for example by combining wall pairs in various positions. This approach has a price. To approximate an arbitrary function, we can cover the input space with a grid, cutting each of the d axes into n pieces and placing one tower per cell. This requires n^d towers, so the number of units grows exponentially with the input dimension.

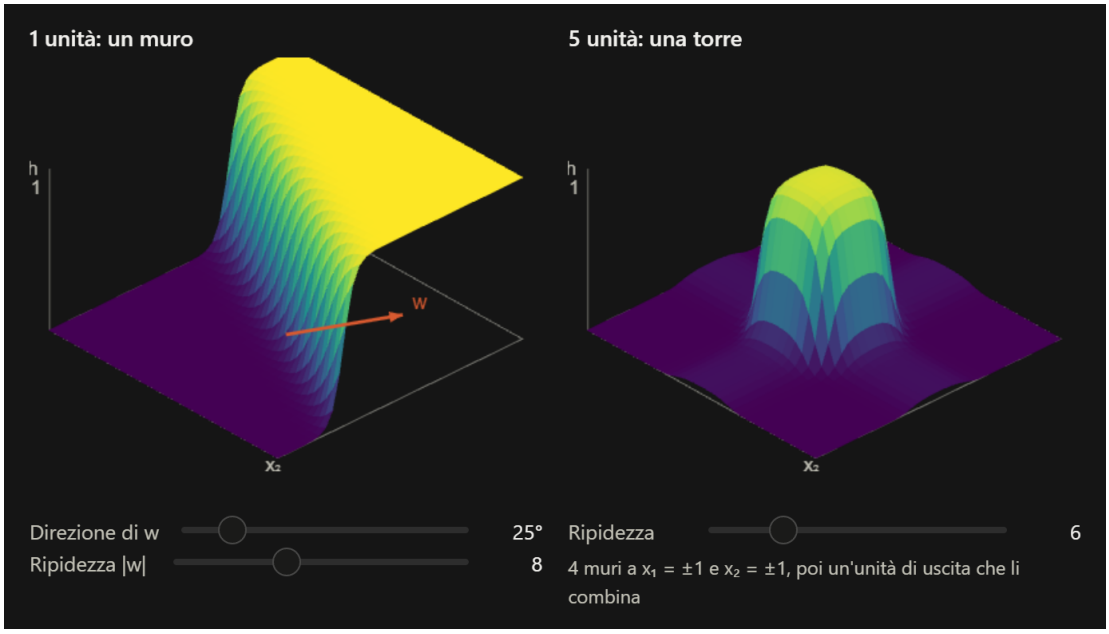


Figure 3.19: 3d surface plot

3.7.2 Two sigmoid units in 2D

to build intuition for the tower construction above, we have to explain how two sigmoid units combine together to define a partially bounded region (two units are never enough to enclose a region). First of all, each unit cuts the plane (Fig 3.20): $\sigma(w_1x_1 + w_2x_2 + b)$ is:

- 0.5 on the line $w_1x_1 + w_2x_2 + b = 0$
- $<$0.5 on one side and near to 0 far from the line (the unit goes off)
- $>$0.5 on the other side and near to 1 far from the line (the unit is on)

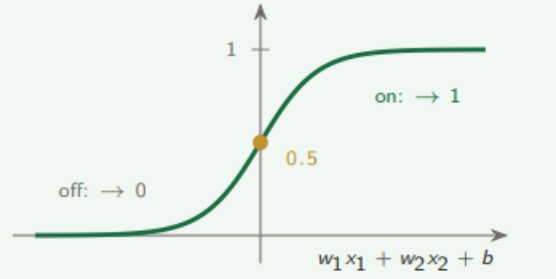


Figure 3.20: sigmoid unit

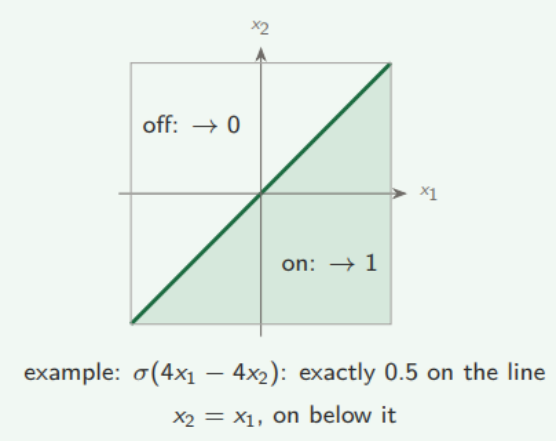


Figure 3.21: sigmoid unit

So computing the output ($\hat{y} = v_A A + v_b B$) by combinating two hidden units we can find three types of combination: *Band, Curved Corner and Wedge.*
DA SPOSTARE The final network output $\hat{y} = \sum_i v_i h_i$ represents a continuous score rather than a probability. Classification is achieved by comparing this score against a decision threshold (e.g., 0.5 for a band or wedge, and 1.5 for a corner). Consequently, the output layer's bias acts as the negative of this threshold, satisfying the condition bias = −threshold. **DA SPOSTARE**

Band

The band is obtained when both units ignore one of the two inputs, the bias indicate where the lines are, the band comes out by the final weights that are opposed $v_A = -v_B$. In Fig 3.22 x_2 has weight 0 so in the 2D-space of the input the sigmods are two vertical lines (Fig 3.23) and the score $\hat{y}$ is shown in Fig 3.24.

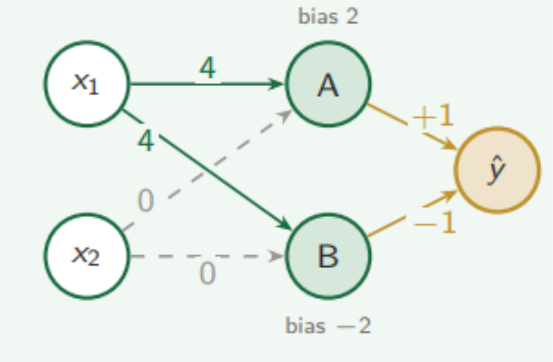


Figure 3.22: Band threshold = 0.5

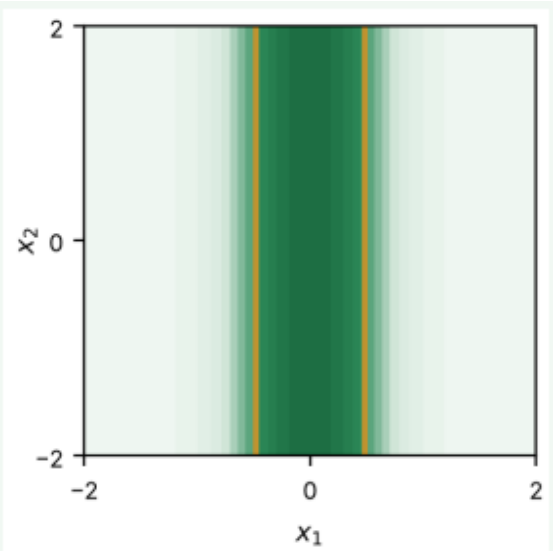


Figure 3.23: Band $\sigma(4x_1 + 2) - \sigma(4x_1 - 2)$

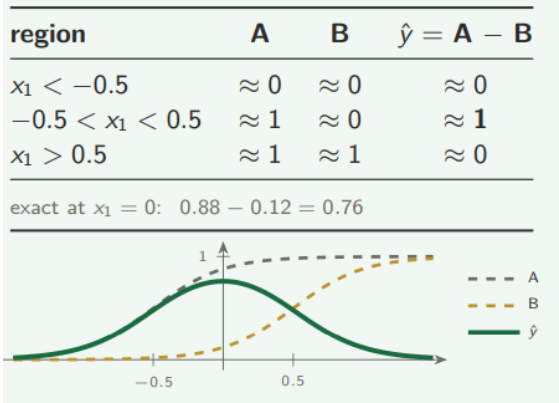


Figure 3.24: Cross section along x_1 with the 1D bump

Curved Corner

The curved corner happens when we add the two crossing lines at the origin that split the plane into four sector, as shown in Fig 3.26. Here the decision threshold is 1.5 because we predict 1 when both units are on (right sector). It's *curved* because in the origin $\hat{y} = 1 < 1.5$

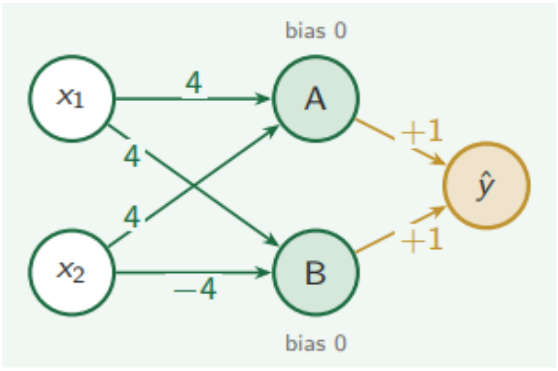


Figure 3.25: Curved Corner threshold 1.5

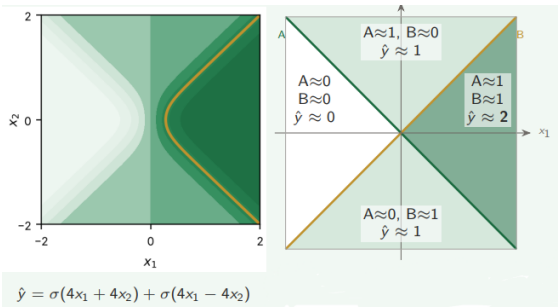


Figure 3.26: Curved Corner

Wedge

The wedge is similar to the curved corner, the difference is on the hidden units weigth as shown in Fig 3.27 infact here the two lines are subtracted (threshold 0.5)

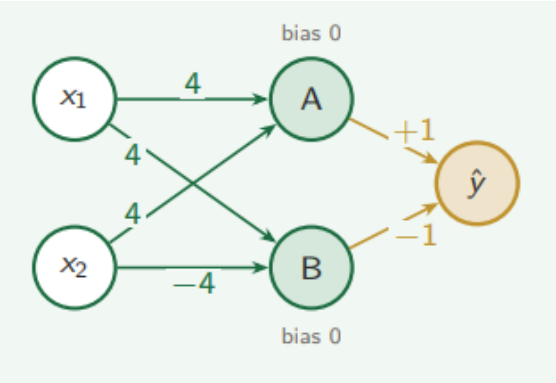


Figure 3.27: Wedge in the nn

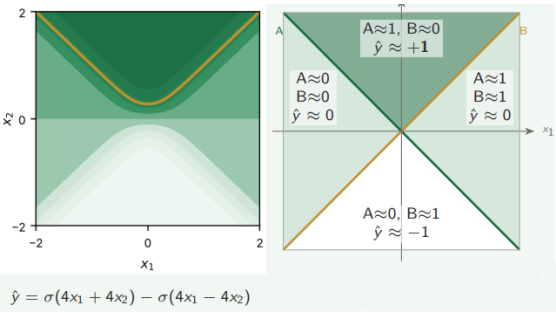


Figure 3.28: Wedge

3.7.3 Deep vs Shallow

We said that a network with a single layer is sufficient to represent any function, but the layer may be infeasibly large. In many circumstances, using deeper models can reduce the number of units required to represent the desired function and can reduce the amount of generalization error.

Definition 5 (The Exponential Capacity of Depth) *A ReLU network with d inputs, L layers of q units can split the input space into about*

$$\left(\frac{q}{d}\right)^{d(L-1)} q^d$$

linear regions: exponential in the depth L.

For example in Figure 3.29 we see that with $t(x) = 2max0, x - 4max0, x - \frac{1}{2}$ two ReLUs make a tent on [0,1], $t(t(x))$ **folds** the tent into two tents and so on.

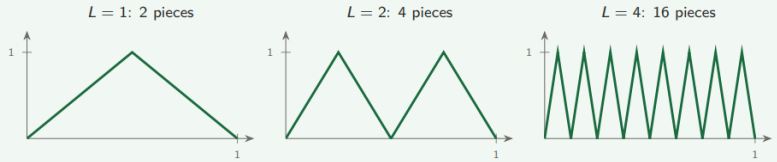


Figure 3.29: Exponential advange of deeper rectifier networks

Figure 3.30 shows a geometric representation of this concept, an absolute value unit $g(z) = |z|$ gives the same output for a point and its mirror image across the line z = 0, it folds the plane like a sheet of paper. Whatever the next layers compute is repeated on both halves; each further layer folds again: 2, 4, 8, . . . copies.

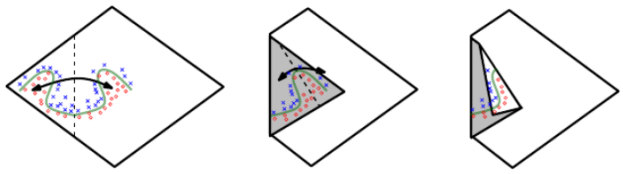


Figure 3.30: Exponential advantage of deeper rectifier networks

A deep model assumes that the target is a composition of simpler functions or has a repeated structure, using the fold concept, depth can be exponentially more efficient.

Chapter 4

Optimization Algorithms

Chapter 5

Notation

To maintain consistency throughout this document, we adopt the standard notation summarized below, largely following standard deep learning literature:

Symbol	Description
$x \in \mathbb{R}^d$	one input, with d features
y (or $\mathbf{y}$)	its target: a number, a class, or a vector
f^*	the unknown true function, $y = f^*(x)$
n	number of training examples
$(x^{(i)}, y^{(i)})$	the i-th training example
$X \in \mathbb{R}^{n \times d}$	all inputs, one example per row
m	minibatch size (examples per update)
k	number of classes

Table 5.1: Data notation summary.

Symbol	Description
L	number of layers (the depth)
d_l	number of units in layer l (its width); $d_0 = d$
$W^{(l)}, b^{(l)}$	weight matrix ($d_{l-1} \times d_l$) and bias vector of layer l
$z^{(l)}$	pre-activation: $W^{(l)\top} h^{(l-1)} + b^{(l)}$
$h^{(l)} = g(z^{(l)})$	activation of layer l; $h^{(0)} = x$
g	activation function of hidden units
o	output function (identity, sigmoid or softmax)
θ	all weights and biases together

Table 5.2: Network notation summary.

Note on indexing: Superscripts in parentheses, such as $^{(l)}$ or $^{(i)}$, always denote indices (referring to layers or training examples, respectively) and are never used as mathematical powers. Vectors are represented in bold lowercase letters, matrices in bold uppercase, and scalars in standard font.

Symbol	Description
$\hat{y} = f(x;\theta)$	prediction of the network
$\ell(\hat{y}, y)$	loss on one example
$J(\theta)$	cost: average loss on the training set
$\nabla_\theta J$	gradient: vector of all $\partial J/\partial\theta_j$
η	learning rate (step size)
logit z	the score before the sigmoid or softmax

Table 5.3: Learning and optimization notation summary.

Symbol	Description
$\sigma(z) = \frac{1}{1+e^{-z}}$	logistic sigmoid, values in $(0, 1)$
$\zeta(a) = \log(1 + e^a)$	softplus: a smooth version of $\max\{0, a\}$
$e_y \in \{0, 1\}^k$	one-hot vector: 1 in position y, 0 elsewhere
$\mathbb{I}[\text{cond}]$	indicator: 1 if cond is true, else 0
$\mathbf{1}$	vector of ones
$\odot$	element-wise product

Table 5.4: Functions and symbols notation summary.